

Chapter 6: Architectural Design

Software Engineering — Ian Sommerville (10th Edition)

Part 1 — Study Notes

Topics Covered

#	Topic
1	What is Architectural Design?
2	Agility and Architecture
3	Examples of Architecture (ATM & Packing Robot)
4	RE & Design Process Overlapping
5	Architectural Abstraction (Small vs Large)
6	Why Software Architecture is Important?
7	Advantages of Explicit Architecture
8	Architectural Representations
8b	UML vs ADL vs Informal Diagrams
9	Architectural Design Decisions
10	Architecture & System Characteristics (NFRs)
11	Conflicting Critical Requirements
12	Architectural Views (4+1 View Model)

Topic 1: What is Architectural Design?

Architectural design is the **first stage in the software design process**. It acts as the **critical link between requirements engineering and implementation**.

It is concerned with two main things:

- **Understanding** how a software system should be organized
- **Designing** the overall structure of that system

Output: An **Architectural Model** — describes how the system is organized as a set of communicating components (modules, subsystems, services, etc.)

Where it sits in the process:

Requirements Engineering → Architectural Design → Detailed Design → Implementation

Key Takeaway: It identifies the main structural components and the relationships between them — the blueprint before building begins.

Topic 2: Agility and Architecture

Even in agile development, it is **generally accepted** that you should design an overall system architecture in the **early stages** of the project.

Key distinction:

- **Refactoring code** = relatively easy ✓
- **Refactoring architecture** = very expensive ✗ (requires changing many components)

Agile doesn't mean skip the architecture. It means: design the architecture early, keep it solid, then be flexible within it.

Key Takeaway: Architecture is the skeleton — you can change muscles and skin (code/features) freely, but restructuring the skeleton late is painful and costly.

Topic 3: Examples of Architecture

Example 1: ATM Machine

Components: Validate via Fingerprint → Validate Card via PIN → Select Options → Cash Dispensed / View Balance → Perform Transaction → Send Command to Database

Example 2: Packing Robot Control System

Objects arrive on conveyor → Robot classifies the object → Packages accordingly → Places on another conveyor for shipment

Key Takeaway: Both examples show architecture as: breaking a system into meaningful components, defining what each does, and how they interact. Common pattern: Input → Process → Output.

Topic 4: RE & Design Process Overlapping

In practice, the **requirements specification process overlaps with architectural design**. They are not strictly sequential.

Why can't they be fully separated?

Because when gathering requirements, you inevitably start thinking about design. The specification always includes some design information whether you intend it or not.

What to do during RE:

- Identify the main architectural components (reflect high level features)
- Show an abstract system architecture depicting subsystem associations
- Use this decomposition to discuss and fine-tune requirements with stakeholders

Key Takeaway: RE and Architectural Design inform and refine each other throughout early development — they are not strictly sequential.

Topic 5: Architectural Abstraction

Level	Focus	Example
Architecture in the Small	Internal structure of ONE complete application	How the ATM app is broken into modules
Architecture in the Large	How MULTIPLE systems/applications connect	ATM + Bank Server + Payment Gateway + Verification
Detailed Design	Individual functions/modules/classes	Authentication function inside ATM

Key Takeaway: Small = one application's internal structure. Large = multiple applications/systems and how they interact. These are about FOCUS, not physical size.

Topic 6: Why Software Architecture is Important?

The system architecture **directly affects the Non-Functional Requirements (NFRs)** of the system.

NFR Types:

- **Functional Requirements (FRs)** — What the system should do (e.g., ATM should dispense cash)
- **Non-Functional Requirements (NFRs)** — How well it does it (performance, security, availability, maintainability)

The way you structure your system determines how fast it runs, how reliable it is, how easy it is to update, and how secure it is.

Key Takeaway: You can't meet your NFRs without well thought out architecture. Architecture IS the foundation for non-functional quality.

Topic 7: Advantages of Explicit Architecture

Advantage	Benefit
Stakeholder Communication	High-level presentation; used as focus of discussion; common ground for all stakeholders
System Analysis over NFRs	Analyze whether system can meet NFRs before writing code; catch issues early
Large-Scale Reuse	Same architecture reused for similar applications; saves redesign time

Key Takeaway: Designing architecture explicitly saves time, improves communication, and ensures quality — all before a single line of code is written.

Topic 8: Architectural Representations

Method 1: Informal Block Diagrams (Box & Line)

- Components represented as **Boxes**
- Sub-components as **Nested Boxes**
- **Arrows** depict data and control signals flowing between components

Drawbacks: Very abstract — does not show nature of relationships or externally visible properties of subsystems.

However useful for: Stakeholder communication, project planning, quick high-level understanding.

Method 2: ADL (Architecture Description Language)

- Globally recognized (UML, ArchiMate)
- **Drawbacks:** Time consuming and expensive

Notation	Detail Level	Speed	Best For
Box & Line Diagrams	Low	Fast	Stakeholders, planning, high-level design
ADL / UML	High	Slow	Documentation, detailed design

Topic 8b: UML vs ADL vs Informal Diagrams

UML Issues: Designed for OO systems → too implementation-oriented, not abstract enough, increases time. Useful only for detailed design.

ADL Issues: Too domain specific, enforces strict rules, not widely used.

Sommerville's Recommendation: Use informal Box/Line diagrams for architecture design — simple, fast, abstract, understood by everyone.

Key Takeaway: No one perfect notation — choose based on audience and required detail level.

Topic 9: Architectural Design Decisions

Architectural design is a **creative process** to develop a system fulfilling both FRs and NFRs.

No formulaic process — depends on:

- System type • Background & experience of architect • Specific requirements

It is a **series of decisions to be made**, not a sequence of activities to be performed.

Common questions every architect faces:

- Distributed or centralized? • How to decompose into components? • How will components communicate? • What patterns are suitable? • How to handle failures? • Buy or build?

Important: Certain fundamental decisions always come up regardless of system type, and they always impact NFRs.

Key Takeaway: Architectural design is not a step-by-step recipe — it's a creative decision-making process guided by experience, system type, and requirements.

Topic 10: Architecture & System Characteristics (NFRs)

NFR	Key Architectural Decisions
Performance	Localize critical operations on same machine; avoid distributed systems; use large (not fine-grain) com

Security	Use layered architecture; critical assets in inner layers; e.g., multi-tier architectures
Safety	Localize safety-critical features in few subsystems; include separate shutdown system; reduces cost of failure
Availability	Include redundant components; allow replacement/update without shutdown; e.g., fault-tolerant architectures
Maintainability	Use fine-grained components; avoid shared data structures (static variables); avoid nesting of components

Key Takeaway: Your architectural decisions must be driven by which NFRs are most critical — there's no one-size-fits-all architecture.

Topic 11: Conflicting Critical Requirements

Some NFRs naturally contradict each other:

NFR	What it demands	Conflict
Performance	Fewer, larger components	↕ Direct conflict
Maintainability	More, smaller components	↕ Direct conflict
Security	More layers (slows things down)	Conflicts with Performance

Two Solutions:

1. **Compromise** — Sacrifice the less critical requirement; prioritize based on stakeholder needs
2. **Multiple Architectural Styles** — Use different styles for separate parts of the system

Key Takeaway: In real systems, NFRs will conflict. The architect must either compromise on less critical requirements or mix architectural styles for different system parts.

Topic 12: Architectural Views — 4+1 View Model (Krutchen, 1995)

No single diagram can show all perspectives. Present **multiple views** to show different perspectives of the architecture.

View	Shows	Useful For
Logical View	Key abstractions as objects/classes; relates requirements to devices	Developers
Process View	Runtime interaction between processes; NFRs like performance, availability	Performance Analysts
Development View	Breakdown of software assigned to teams/developers	Project Managers & Developers
Physical View	Hardware & software distribution across processors	System Engineers
Conceptual View (+1) (Hofmeister et al., 2000)	Abstract high-level requirements; product line architecture for future products	Future Product Lines

The **4 views are linked via Use Cases** — use cases describe scenarios that drive the architecture and act as the glue connecting all views.

Key Takeaway: No single diagram captures everything — use multiple views to communicate different aspects to different stakeholders.

